DSAP

Laboratory Manual

LAB 03

Discrete-Time Convolution and LTI System Properties using MATLAB

	DSAP
Lab Title	Discrete-Time Convolution and LTI System Properties using MATLAB
Prerequisites	• Basic programming concepts • Lab 02 — Discrete-Time Signal Generation and Manipulation in MATLAB
Software	MATLAB R2020a or later

1. Lab Objectives

By the end of this lab, students will be able to:

- Implement nested for loops by writing a custom discrete convolution engine. in MATLAB.
- Mathematically account for non-zero starting times and track time-axis boundaries in signal vectors. .
- Validate fundamental Linear Time-Invariant (LTI) system properties (Commutative, Associative, and Distributive) empirically.
- Visualize how different signal interactions (pulses, exponentials, and steps) alter the output shape.
- Implement a loop-driven moving average filter to perform practical data-smoothing tasks over a noisy signal.

Part A — Discrete-Time Convolution

Background

Mathematically, convolution "mixes" one signal with another. If you have an input signal x[n] and a system's impulse response h[n], the discrete-time convolution of the two; denoted by the asterisk (*); gives the output signal y[n] via the summation:

$$y[n] = \Sigma\ x[k] \cdot h[n - k] \quad , \quad \text{summed over all } k$$

To visualize this, imagine the **"Flip and Slide"** concept:

1. **Flip**: Take the system response h[k] and flip it in time to get h[-k].
2. **Shift (Slide)**: Shift the flipped signal by the index n to get h[n - k].
3. **Multiply**: Multiply the input signal x[k] by the shifted, flipped signal h[n - k].
4. **Sum**: Calculate the sum of the product terms. This total value is the output y[n] for that specific moment in time.

The total output length of a discrete convolution is always:

$$\text{Length}(y) = \text{Length}(x) + \text{Length}(h) - 1$$

Task 1: Convolution using loop vs Built-in function

x[n] = [1, 2, 3, 4] for n = 0, 1, 2, 3
h[n] = [2, 1, 2] for n = 0, 1, 2,

1. *Initialize a vector y_manual to zeros with the correct total output length.*
2. *Use nested for loops to compute the manual discrete convolution sum. The outer loop runs through every index of the output vector, while the inner loop iterates through the indices of x.*
3. *Inside the inner loop, use an if statement to check that the calculated index for h is valid within MATLAB's 1-based indexing limits before performing multiplication.*
4. *Compute the same convolution using MATLAB's built-in function: y_built = conv(x, h).*
5. *Create a single figure with two subplots using stem. Subplot 1: Your manual loop result. Subplot 2: MATLAB's conv result. Label axes and enable grids.*

Task 2: Managing shifted time origins

Adapt your time-axis tracking to handle sequences that do not start at index n=0.

x[n] = [3, -1, 2] for n= -1, 0, 1
h[n] = [1, 2, 1, 4] for n= -2, 3, 4, 5

1. Define the precise time vectors nx and nh
2. Calculate the boundaries of the output time vector ny analytically:
   - ny_start = nx(1) + nh(1);
   - ny_end = nx(end) + nh(end);
3. Construct the final time array: ny = ny_start : ny_end;
4. Compute the output sequence y using conv(x, h)
5. Plot x[n], h[n], and y[n] in a single figure using 3 subplots. Ensure each stem plot explicitly maps to its exact time index vector (nx, nh, or ny) on the X-axis.

Task 3: Verifying the Commutative Property

*Prove experimentally that the order of signals in a convolution operation does not matter (x * h = h * x).*

1. Using the signals from Task 2, compute y1 = conv(x, h) and y2 = conv(h, x)
2. Evaluate if the sequences are mathematically identical using an if-else block evaluating if max(abs(y1 - y2)) < 1e-12 or if isequal(y1, y2).
3. Overlay both results on a single plot using different marker styles (e.g., circles 'o' for y1, crosses 'x' for y2) to visually confirm they occupy the exact same spatial coordinates.

Part B — Fundamental Signal Interactions

Task 4 : The Identity and Shifting Property of the Impulse

Understand how an LTI system behaves when the input is an ideal unit impulse or a shifted impulse.

h[n] = [0.5, 1,-0.5, 2] for n= 0, 1, 2, 3

1. Convolve h[n] with a unit impulse delta[n] = [1] at n=0
2. Convolve h[n] with a shifted impulse delta[n-3]. (Represent this input vector as [0, 0, 0, 1] if starting from n=0). Alternatively, use function impseq in Lab 2.
3. Plot h[n], the result of the first convolution, and the result of the second convolution in a single figure using subplots. Track the time axes carefully using the shift summation rules from Task 2.

Task 5: Rectangle meeting Rectangle

Observe how the relative width of two square pulses determines whether the convolution output is a perfect triangle or a trapezoid.

1. Case 1 (Equal Lengths): Define two rectangular pulses using
   - x[n] = 1 for $0 \le n \le 5$ (Length 6)
   - h[n] = 1 for $0 \le n \le 5$ (Length 6)
   - Compute their convolution
2. Case 2 (Unequal Lengths):
   - h[n] = 1 for $0 \le n \le 2$ (Length 3)
   - Compute the convolution
3. Plot the input signals and the resulting output shapes side-by-side to visibly note the structural transition from a sharp triangle to a flat-topped trapezoidal plateau.

Task 6: Step Response of a First-Order System

Observe how an exponential decay system responds to a sudden, sustained constant input (Unit Step).

Given System & Input:

Impulse Response: $h[n] = (0.7)^n u[n]$ for $0 \le n \le 15$

Input signal: $x[n] = u[n]$ for $0 \le n \le 15$

1. Compute y[n] = x[n] * h[n] via conv
2. Plot all three signals (x[n], h[n], and y[n]) using subplots to track how the output gradually climbs and stabilizes.

Part C — System Properties & Structural Layouts

Task 7: Distributive Property Verification

Prove that processing a signal through two parallel systems added together is equivalent to adding their individual outputs: $x * (h_1 + h_2) = (x * h_1) + (x * h_2)$.

Given sequences:

$x[n] = [1, 2, 1]$, $h_1[n] = [1, -1]$, $h_2[n] = [2, 1]$

1. Calculate the Left-Hand Side (LHS): y_left = conv(x, h_1 + h_2); (Note: Since h_1 and h_2 have identical length, they can be added directly).
2. Calculate the Right-Hand Side (RHS): y_right = conv(x, h_1) + conv(x, h_2)
3. Plot both y_left and y_right in a single window using hold on. Use distinct marker styles (e.g., 'o' and 'x') to visually verify that both processing tracks yield identical data points.

Quick Reference — MATLAB Syntax List

Task	MATLAB Code
Create time vector (continuous)	`t = 0 : 0.001 : 1;`
Create discrete index	`n = 0 : 1 : 50;`
Evenly spaced points	`t = linspace(0, 1, 1000);`
Generate sine wave	`x = sin(2*pi*f*t);`
Generate cosine wave	`x = cos(2*pi*f*t);`
Unit step (continuous)	`u = (t >= 0);`
Unit step (discrete)	`u = (n >= 0);`
Unit impulse (discrete)	`delta = (n == 0);`
Ramp signal	`r = t .* (t >= 0);`
Sinc function	`x = sinc(t);`
Decaying exponential	`x = exp(-a*t);`
Complex exponential	`x = exp(1j * omega * t);`
Real part	`real(x)`
Imaginary part	`imag(x)`
Magnitude	`abs(x)`
Continuous plot	`plot(t, x, 'b', 'LineWidth', 1.5);`
Discrete plot	`stem(n, x, 'filled');`
Multiple subplots	`subplot(rows, cols, index);`
Add title	`title('Signal Name');`
Add x label	`xlabel('Time (s)');`
Add y label	`ylabel('Amplitude');`
Turn grid on	`grid on;`
Set axis limits	`axis([xmin xmax ymin ymax]);`
Add legend	`legend('Signal 1', 'Signal 2');`